Lab 5: Login, bcrypt, and sessions

This lab session is dedicated to using sessions and hashed password in your project. We will continue with the "projects and skills" database in this lab. To try and test some code, you can use the data I provide for the labs, but in the end, the database of your project MUST use **YOUR (OWN) DATA**!

1) Hashed password

During lab 4, you created a login form and displayed the information received on the web page. Today, we will process the information, test if it is valid, and react accordingly. We need to define an admin login and password (as clear text for now) in our code, it must be done in the variable and constant decalarations.

```
//--- DEFINE VARIABLES AND CONSTANTS
const port=8080
const app=express()
const adminPassword='wdf#2025'
```

We want to hash the password using the bcrypt package. You must install the bcrypt package, add it to the packages loaded at the beginning of your "server.js" file.

```
const bcrypt=require('bcrypt')
```

We will hash this password using a function (you can put it at the end of your server.js code):

```
function hashPassword(pw, saltRounds) {
    bcrypt.hash(pw, saltRounds, function(err, hash) {
        if (err) {
            console.log('---> Error hashing password:', err);
        } else {
            console.log(`---> Hashed password: ${hash}`);
        }
    });
}
```

This function takes two parameters: pw and saltRounds. The first parameter is the cleartext password to hash and the second one is the salt (number of rounds) you want to use to hash the password. This value goes from 1 to 20, here, we will use 12 which is a standard secure value (20 being the maximum security but takes longer time to generate the hash).

You must call the function (only once), I recommend to call it when the server listens to the port:

```
//--- LISTEN TO INCOMING REQUESTS
app.listen(port, () => {
    hashPassword("wdf#2025", 12); // hash the password "wdf#2025"
    console.log(`Server running on http://localhost:${port}`)
})
```

Copy/paste the hashed password (only once).

```
PS C:\Users\lanjer\Desktop\box\ju2025\teaching\courses\semester1\lp1\wevdevfun\code\lecture-sessions
\portfolio-jl-03-sessions> node .\server.js
Server running on http://localhost:8080
---> Hashed password: $2b$12$p5.UuPb9Zh.siIc78Ie.Nu9eGx9d5OLT2pkecedig2P.6CdfL1ZUa
```

```
//const adminPassword='wdf#2025'
const adminPassword='$2b$12$p5.UuPb9Zh.siIc78Ie.Nu9eGx9d5OLT2pkecedig2P.6CdfL1ZUa'
```

Now that we have the hashed password, we can use the bcrypt.compare() function in our code to test if the password coming from the form is the correct one.

```
//--- LOGIN FORM
app.get('/login', (request, response) => {
    response.render('login') // the login form to send to the client
})
//--- LOGIN PROCESSING
app.post('/login', (request, response) => {
    // the treatment of the data received from the client form
    console.log(`Here comes the data received from the form on the client: ${request.body.un} - ${request.
    body.pw} `)
    if (request.body.un==="admin") {
        bcrypt.compare(request.body.pw, adminPassword, (err, result) => {
            if (err) {
                console.log('Error in password comparison')
                model = { error: "Error in password comparison." }
                response.render('login', model)
            }
            if (result){
                request.session.isLoggedIn=true
                request.session.un=request.body.un
                request.session.isAdmin=true
                console.log('---> SESSION INFORMATION: ', JSON.stringify(request.session))
                response.render('loggedin')
            } else {
                console.log('Wrong password')
                model = { error: "Wrong password! Please try again." }
                response.render('login', model)
            }
        })
    } else {
        console.log('Wrong username')
        model = { error: "Wrong username! Please try again." }
        response.render('login', model)
    }
})
```

In the previous code, we want to check if the username is “admin”, if not we build a model with an error information and send it to the login view. In the “login.handlebars” file, use a {{#if error}} … {{/if}} condition to print the received error message.

If the username is correct (is “admin”), then we compare the password coming from the form to the adminPassword (hashed password) variable. If they are different, we build a model with an error text and send it to the login view. If they are the same, we save the information in the **session**: the username in “session.un”, both “session.isLoggedIn” and “session.isAdmin” are set to true.

2) Using sessions

When the user is connected, we want to save this information until disconnection. Because http is a **stateless** protocol, every request/response is independent. It means we cannot **SAVE** information using http ONLY. We need another way to memorize information: **sessions**. A session is (Brave Browser Integrated AI, 2024): **“HTTP Session Definition**

In HTTP (Hypertext Transfer Protocol), a session refers to **a sequence of multiple request-response transactions between a client (usually a web browser) and a server**, where the server maintains context information about the client’s activities across multiple requests. This context information is stored on the server-side and is associated with a unique identifier, known as a session ID or session token.

A session is necessary when a client interacts with a server in a way that requires the server to remember certain information about the client’s previous requests, such as:

1. User authentication: The server needs to remember the user's login credentials or authentication token.
2. Shopping cart: The server needs to store the items a user has added to their cart across multiple page requests.
3. Form data: The server needs to retain form data, such as input fields, across multiple requests.

To establish a session, the server typically:

1. Generates a unique session ID (e.g., a random string or a cryptographic token).
2. Sends the session ID to the client as an HTTP cookie (usually with a name like JSESSIONID, PHPSESSID, or ASPSESSIONID).
3. Stores the session ID and associated context information on the server-side (e.g., in a database or memory cache).

When the client makes subsequent requests, it includes the session ID in the HTTP headers or as a cookie value. The server uses this session ID to retrieve the associated context information and continue the conversation with the client.**"**

Let's go with sessions! To use sessions, you need to install and add to your code the "express-session" package. We also want to automatically save the session ids and session variables into a database to simplify our code and the management of sessions.

```
const session=require('express-session')
const sqlite3=require('sqlite3') // load the sqlite3 package
const connectSqlite3 = require('connect-sqlite3') // store the sessions in a
SQLite3 database file
```

```
///--- CONNECT TO DATABASE
const dbFile='my-project-data.sqlite3.db'
db=new sqlite3.Database(dbFile)
//--- STORE SESSIONS IN THE DATABASE
const SQLiteStore = connectSqlite3(session) // store sessions in the database
//--- DEFINE THE SESSION
app.use(session({ // define the session
    store: new SQLiteStore({db: "session-db.db"}),
    "saveUninitialized": false,
    "resave": false,
    "secret": "This123Is@Another#456GreatSecret678%Sentence"
}))
```

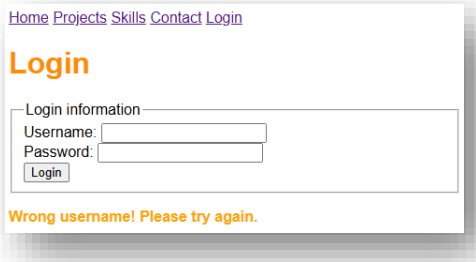
The secret part can be anything you want (a non-empty random string is better), it is a string used to add secret and random seed in your sessions' keys. This code will create a "session-db.db" file to store the sessions in your project directory.

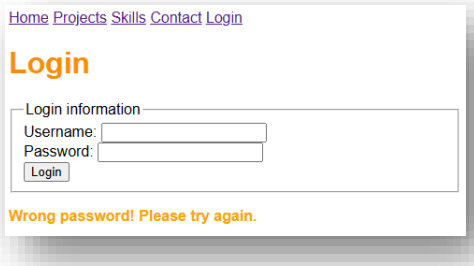
Perfect, the session variables are defined and stored in the databse. Now, we want them to be available on any part of our single page application, so we need to add some middleware code to make them visible from anywhere on our web site.

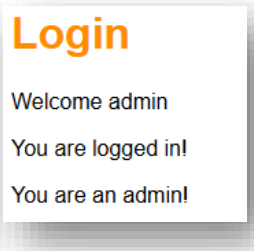
```
//--- MAKE THE SESSION AVAILABLE IN ALL HANDLEBAR FILES AT ONCE!
app.use((request, response, next) => {
    response.locals.session = request.session
    next()
})
```

This middleware code will automatically make the session visible on any part of our site by copying the request session into the response locals (response.locals). Now you can use the "session.**variable**" in any handlebars file where **variable** is the name of the session variable you want to use!

In your loggedin page, you can display the message that the user is loggedIn and add the information if it is an “admin” using “session.isLoggedIn” and “session.isAdmin”:





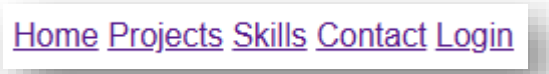


Remark: if you plan for grades 3 or 4, you don’t need to have a user table with your admin information: login and hashed password; you can use this code that test the login and password in your “server.js” file (hardcoded verification). However, if you plan for grade 5, you MUST have a table in your database containing the list of users. In other words, you must use a “User” table in your database, it means that the information: login, password, and isAdmin must come from your database.

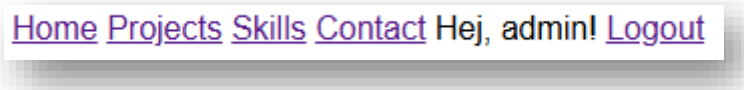
Please, run the server and test it! You should obtain the information in the session, similar to:

```
PS C:\Users\lanjer\Desktop\box\ju2025\teaching\courses\semester1\lp1\wevdevfun\code\lecture-sessio
ns\portfolio-jl-03-sessions> node .\server.js
Server running on http://localhost:8080
---> Hashed password: $2b$12$MSZYIUrUoVJL8H8IafaRPeMSb/X85qEYewpjjtBrh0CZ3EyrvMoH.
Here comes the data received from the form on the client: admin - wdf#2025
---> SESSION INFORMATION:  {"cookie":{"originalMaxAge":null,"expires":null,"httpOnly":true,"path":
"/"},"isLoggedIn":true,"un":"admin","isAdmin":true}
```

You must modify your menu to make it simple with the Login link when the user is NOT connected:



And add a nice welcome text and the Logout link when a user is connected:



Then, of course, you need to define the “/logout” route in your routes:

```
//--- LOGOUT PROCESSING
app.get('/logout', (req, res) => {
    req.session.destroy( (err) => { // destroy the current session
        if (err) {
            console.log("Error while destroying the session: ", err)
            res.redirect('/') // go back to homepage
        } else {
            console.log('Logged out...')
            res.redirect('/') // go back to homepage
        }
    })
})
```

Please, modify your menu to print the correct information depending on the connection of a user:

```
{{#if session.isLoggedIn}}
    Hej, {{session.un}}!
    <a href="/logout">Logout</a>
{{else}}
    <a href="/login">Login</a>
{{/if}}
```

Then, you do not need to pass the session anymore in your model to your handlebar templates, they will know it from the saved session! The code is simpler, you just need to use session.AAA to get access to the session variable AAA in your code. Brilliant! :)

If you have time, you can start implementing your CRUD on ONE table. This will be next week session!

Please, remember that:

Web Dev is **Fun**

:)